Thesis Research: Capacity planning on docker containers and other comparative studies

Introduction and motivation:

The aim of this topic is to study the impact of resource allocation (e.g. the number of CPUs or cores) on the performance (such as latency) of docker containers and explore how capacity planning concepts can be made more practical and teachable through hands-on experiments using Docker containers that they can do with their own computer. Qualitatively, we know that the higher number of cores will result in a lower latency. However, we would like to perform quantitative prediction by building models that can predict the performance of docker based on its workload and resource specifications. We aim to design the workload for the applications to be run in Docker - perform measurements in Docker to obtain basic workload parameters such as CPU access rate and CPU usage distribution - create and validate a discrete-event simulation model, which uses the measurements and resource specification, to predict the performance. Traditional courses such as COMP9334 focus heavily on theory and queueing models but lack real performance measurement. To bridge this gap, this work sets up a simple containerized environment where CPU, memory, and workload levels can be varied to observe their effects on latency and throughput.

Tools such as cAdvisor and Vegeta are used to collect and visualize performance metrics, demonstrating how theoretical service-demand models relate to real system behavior. The goal is to create a platform that students can run on their own machines to explore capacity planning the same way it is practiced in industry — through measurement, experimentation, and modelling rather than theory alone.

Background:

Queueing theory:

Queueing theory is the mathematical framework used to model systems where requests (jobs, packets, or users) compete for shared resources. It quantifies metrics like *response time (W)*, *throughput (X)*, and *utilisation* ($\rho = \lambda/\mu$) by assuming probabilistic arrivals and services.
In capacity planning, queueing models—such as the M/M/1 or M/M/c queues—help predict how latency increases non-linearly as utilisation approaches saturation [4]. This allows performance engineers to evaluate bottlenecks and predict system behaviour before deployment. In addition to classical queueing models, the experiment aligns with operational laws of system

performance. The utilization law U=X·SU = X \cdot SU=X·S and Little's Law N=X·RN = X \cdot RN=X·R relate the observed metrics—CPU busy time, throughput, and latency—without assuming specific distributions. These relationships form the empirical foundation for validating queueing predictions in containerized environments.

Discrete Event simulation:

Discrete Event Simulation is a computational method for modelling systems where changes occur at distinct time points (events) such as arrivals, departures, or service completions. Each event updates the system's state, allowing complex queueing systems to be studied when analytical solutions are intractable.

In the COMP9334 course, DES is taught using *synthetic theoretical data* (Poisson arrivals, exponential service). However, this project extends that by using real experimental data — collected from container-based web services under controlled load — to drive the simulation.

By feeding empirical distributions of arrival rates, service times, and utilization from Docker experiments into the DES model, we can recreate realistic queueing behaviour and validate or refine theoretical predictions. This approach bridges the gap between analytical models and real-world measurements, offering students a clearer understanding of how theoretical assumptions map to operational systems.

Capacity planning:

Capacity planning is the process of determining the resources (CPU, memory, instances, or network bandwidth) required to maintain acceptable performance under varying demand. It sits at the intersection of queueing theory, simulation, and performance engineering.

Traditional capacity planning methods rely on analytical models or expert heuristics, while modern approaches use monitoring data and machine learning to predict future demand. Research such as **Vitui and Chen (2021)** introduced MLASP, which uses regression and ensemble models to predict system throughput from workload parameters, while **Müller et al. (2022)** applied machine learning to enterprise-scale monitoring data for automatic resource sizing and anomaly detection.

In this thesis, capacity planning is examined through a containerized experimental framework. Using Docker, Prometheus, Grafana, and cAdvisor, the system collects live resource metrics while workloads (generated by Vegeta/wrk2) vary in intensity. This enables a controlled study of performance degradation and resource saturation. The data can then be used either to drive data-informed discrete-event simulations or to train predictive models, providing two complementary approaches to modern capacity planning — analytical (DES) and data-driven (ML).

While industrial capacity-planning systems such as AWS Compute Optimizer, Azure Advisor, and Facebook Prophet represent the current state of practice, they are not ideal for teaching

fundamental principles of performance modelling and capacity planning since they are highly automated alongside needing paid subscription.

1. What are the resource parameters? Are they in Docker?
   -> CPU cores, memory size, replication count, thread pool size, and timeouts

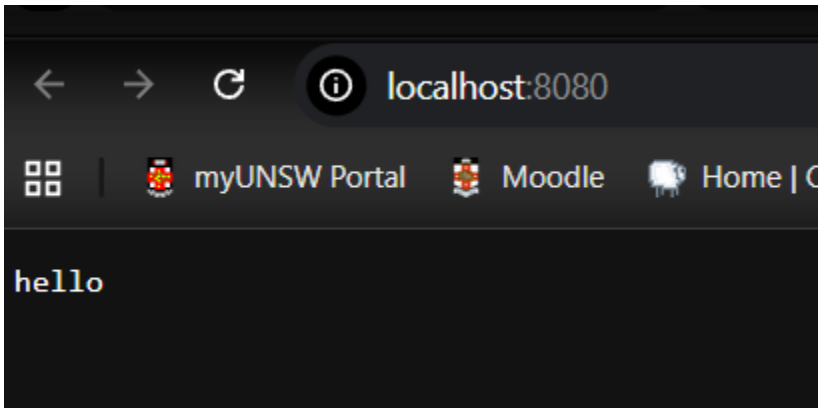
– to do: messaging systems analysis

Literature Survey

Tools:

From my research, **cAdvisor + Prometheus + Grafana** form a standard open-source monitoring pipeline widely used in performance engineering.
GitHub - google/cadvisor: Analyzes resource usage and performance characteristics of running containers.
https://prometheus.io/docs/prometheus/latest/getting_started/
https://github.com/stefanprodan/dockprom
https://docs.docker.com/compose/
https://qainsights.com/deep-dive-into-vegeta-http-load-testing-tool-and-library
https://www.virtualizationhowto.com/2024/10/docker-container-monitoring-with-cadvisor-node-exporter-prometheus-and-grafana/
https://kifarunix.com/monitor-docker-containers-metrics-using-grafana
https://blog.veloscot.uk/container-monitoring
https://prometheus.io/docs/guides/cadvisor/

For my docker experiment, an http-echo server from hasicorp as the main application.
https://hub.docker.com/r/hashicorp/http-echo



It's a minimal HTTP service used to simulate a backend responding to requests.

cAdvisor was the most helpful for me, it collects live metrics for CPU, memory, I/O, and network for each container. I'll be using this for the time being, and look at prometheus and grafana later on since I am not yet used to its UI, however it seems to be a very useful visualisation tool too.

I've also found vegeta useful for load testing on my server:
https://github.com/tsenart/vegeta

Industrial practises:

Use of Docker and its suitability as a teaching resource:

Docker and Resource Management:

Prometheus and Grafana were selected as the core monitoring and visualization tools for this thesis due to their widespread adoption in both academic and real-world DevOps and production monitoring environments. Prometheus provides a highly efficient time-series database and querying model (PromQL) suitable for collecting metrics from containerized systems such as Docker and Kubernetes. Grafana complements Prometheus by offering dynamic dashboarding and visual analytics that make performance trends and bottlenecks easily interpretable.

Recent studies confirm that this open-source pair has become a de facto standard in performance monitoring and DevOps workflows. Kumar *et al.* [1] describe Prometheus + Grafana as "a common monitoring stack used by DevOps teams for storing and visualization of time-series data." Dhane *et al.* [2] similarly highlight that these tools "have gained significant attention" within the open-source monitoring ecosystem. A more recent industrial case study by Lubis *et al.* [3] demonstrates successful deployment of Prometheus and Grafana to improve network-performance visibility in an operational company environment.

By incorporating this stack, the thesis mirrors how performance monitoring and capacity planning are executed in real-world practice rather than relying solely on purely analytical approaches. This directly supports the project's goal of bridging theoretical learning with modern industry practices. I explored some other tools like *ELK (Elasticsearch, Logstash, Kibana)* or *InfluxDB which* are heavier, less integrated with Docker Compose, and more suited to enterprise-scale telemetry, not student or single-host environments.

After forecasting, they use statistical or ML models to predict (using tools such as ARIMA, Prophet, Holt-Winters) [5]. This is used by AWS and Google. Then they use MATLAB or Python for DES. Also, most enterprises offload capacity planning to cloud vendors entirely, such as AWS Cost Explorer + Compute Optimizer, Azure Advisor, etc [6].

Preliminary Results and Preparations:

Docker Experiment:

- Setting up wsl2 if not using a linux machine, and then using Docker Desktop
- I was using wrk2 for workload generation: https://testdev.tools/wrk2/, but switched to Vegeta later on.

Created a repo with 2 files:
-> Docker-compose.yml and prometheus.yml

C: > Sadat_files > Current_Term > Thesis > Capacity_lab > docker-compose.yml

```yaml
services:
  cadvisor:

  ▷ Run Service
  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro

  ▷ Run Service
  grafana:
    image: grafana/grafana-oss:latest
    ports: ["3000:3000"]
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana-data:/var/lib/grafana

volumes:
  grafana-data:
```

C: > Sadat_files > Current_Term > Thesis > Capacity_lab > ! prometheus.yml

```yaml
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: cadvisor
    static_configs:
      - targets: ["cadvisor:8080"]
```

-> Launched it in power shell with command: docker compose up -d
-> It launches the 4 sites:

App: http://localhost:8080

cAdvisor UI: http://localhost:8081

Prometheus: http://localhost:9090

Grafana: http://localhost:3000 (user: admin / pass: admin)

```
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> docker compose up -d
[+] Running 38/38
 ✔grafana Pulled                                                                    72.6s
 ✔cadvisor Pulled                                                                   71.7s
 ✔prometheus Pulled                                                                 59.9s
 ✔app Pulled                                                                         5.9s
[+] Running 6/6
 ✔Network capacity_lab_default         Created                                       0.1s
 ✔Volume "capacity_lab_grafana-data"   Created                                       0.0s
 ✔Container capacity_lab-cadvisor-1    Started                                       1.7s
 ✔Container capacity_lab-prometheus-1  Started                                       1.8s
 ✔Container capacity_lab-app-1         Started                                       1.7s
 ✔Container capacity_lab-grafana-1     Started                                       1.7s
```

```
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> curl http://localhost:8080/


StatusCode        : 200
StatusDescription : OK
Content           : hello

RawContent        : HTTP/1.1 200 OK
                    X-App-Name: http-echo
                    X-App-Version: 1.0.0
                    Content-Length: 6
                    Content-Type: text/plain; charset=utf-8
                    Date: Wed, 15 Oct 2025 03:02:45 GMT

                    hello

Forms             : {}
Headers           : {[X-App-Name, http-echo], [X-App-Version, 1.0.0], [Content-Length, 6], [Content-Type, text/plain; charset=utf-8]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 6
```

-> Then I was having auth issues with wrk2:

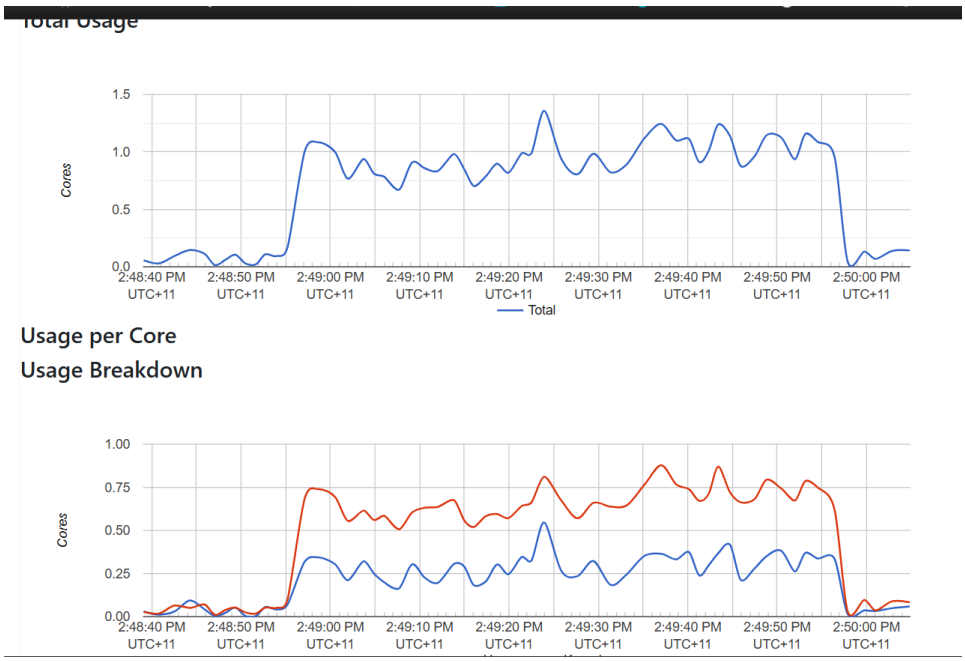
```
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> docker run --rm --network host ghcr.io/giltene/wrk2 -t4 -c64 -d60s -R1000 http://host.docker.internal:80
80/
Unable to find image 'ghcr.io/giltene/wrk2:latest' locally
docker: Error response from daemon: error from registry: denied
denied

Run 'docker run --help' for more information
```

So I switched to Vegeta and ran:
$TARGET = "http://host.docker.internal:8080/"

echo "GET $TARGET" | docker run --rm -i peterevans/vegeta `sh -c "vegeta attack -duration=60s -rate=800 | vegeta report"

```
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> $TARGET = "http://host.docker.internal:8080/"
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> echo "GET $TARGET" | docker run --rm -i peterevans/vegeta `sh -c "vegeta attack -duration=60s -rate=800
| vegeta report"
Unable to find image 'peterevans/vegeta:latest' locally
latest: Pulling from peterevans/vegeta
050382585609: Pull complete
630468763c4c: Pull complete
Digest: sha256:eb65f499cd1b0f1402a56794d7711c49121db6ff8ea7d878513d76601ee0d502
Status: Downloaded newer image for peterevans/vegeta:latest
Requests      [total, rate, throughput]         48000, 800.02, 800.00
Duration      [total, attack, wait]             1m0s, 59.999s, 1.333ms
Latencies     [min, mean, 50, 90, 95, 99, max]  918.718µs, 1.324ms, 1.131ms, 1.451ms, 1.67ms, 3.505ms, 90.936ms
Bytes In      [total, mean]                     288000, 6.00
Bytes Out     [total, mean]                     0, 0.00
Success       [ratio]                           100.00%
Status Codes  [code:count]                      200:48000
Error Set:
```



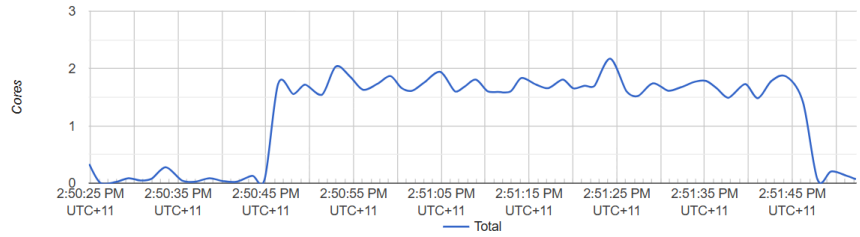
This screenshot is from the cAdvisor page. It seems the application is indeed cpu bound under heavy load.

-> Then I began to vary the rate.

 # 1200 req/s

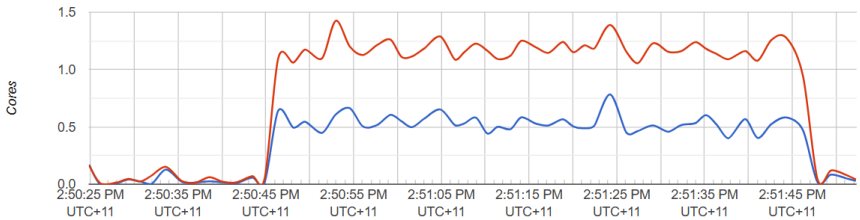
echo "GET http://host.docker.internal:8080/" |
  docker run --rm -i peterevans/vegeta sh -c "vegeta attack -duration=60s -rate=1200 | vegeta report"

Total Usage



Usage per Core

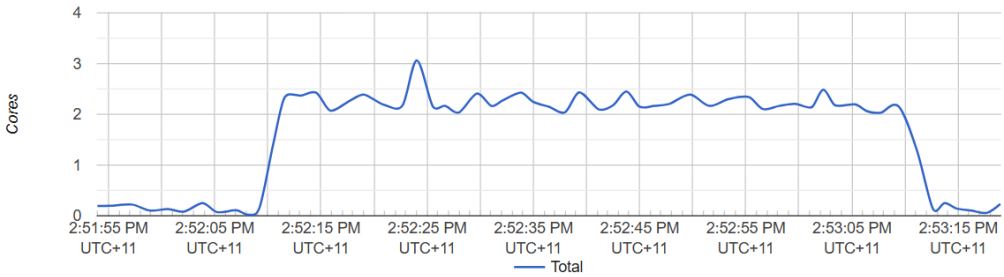
Usage Breakdown



 # 1600 req/s

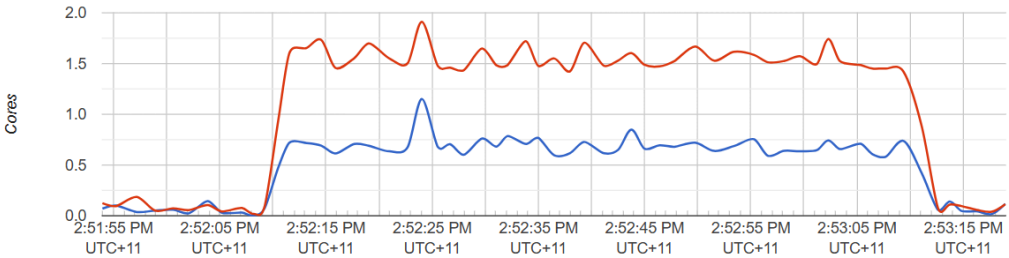
echo "GET http://host.docker.internal:8080/" |
docker run --rm -i peterevans/vegeta sh -c "vegeta attack -duration=60s -rate=1600 | vegeta report"

Total Usage



Usage per Core

Usage Breakdown



```
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> echo "GET $TARGET" | docker run --rm -i peterevans/vegeta `sh -c "vegeta attack -duration=60s -rate=800
| vegeta report"
Requests      [total, rate, throughput]         48000, 800.03, 800.01
Duration      [total, attack, wait]             59.999s, 59.998s, 1.181ms
Latencies     [min, mean, 50, 90, 95, 99, max]  874.844µs, 1.185ms, 1.097ms, 1.325ms, 1.49ms, 2.528ms, 26.849ms
Bytes In      [total, mean]                     288000, 6.00
Bytes Out     [total, mean]                     0, 0.00
Success       [ratio]                           100.00%
Status Codes  [code:count]                      200:48000
Error Set:
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> echo "GET $TARGET" | docker run --rm -i peterevans/vegeta `sh -c "vegeta attack -duration=60s -rate=1200
 | vegeta report"
Requests      [total, rate, throughput]         72000, 1200.02, 1200.00
Duration      [total, attack, wait]             1m0s, 59.999s, 1.206ms
Latencies     [min, mean, 50, 90, 95, 99, max]  907.08µs, 1.287ms, 1.159ms, 1.48ms, 1.648ms, 4.223ms, 24.758ms
Bytes In      [total, mean]                     432000, 6.00
Bytes Out     [total, mean]                     0, 0.00
Success       [ratio]                           100.00%
Status Codes  [code:count]                      200:72000
Error Set:
PS C:\sadat_files\Current_Term\Thesis\Capacity_lab> echo "GET $TARGET" | docker run --rm -i peterevans/vegeta `sh -c "vegeta attack -duration=60s -rate=1600
 | vegeta report"
Requests      [total, rate, throughput]         96000, 1600.02, 1599.98
Duration      [total, attack, wait]             1m0s, 59.999s, 1.355ms
Latencies     [min, mean, 50, 90, 95, 99, max]  922.296µs, 1.518ms, 1.236ms, 1.583ms, 1.86ms, 6.959ms, 93.737ms
Bytes In      [total, mean]                     576000, 6.00
Bytes Out     [total, mean]                     0, 0.00
Success       [ratio]                           100.00%
Status Codes  [code:count]                      200:96000
Error Set:
```

And this was the powershell vegeta report output for all three cases. I tabulated this below:

Request Rate (req/s)	Total Requests	Mean Latency (ms)	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Max Latency (ms)	Success (%)
800	48,000	1.185	1.097	1.325	1.490	2.528	26.849	100
1,200	72,000	1.287	1.159	1.480	1.648	4.223	24.758	100
1,600	96,000	1.518	1.236	1.583	1.860	6.959	93.737	100

-> table: adjust request rate with workload. But not workload per requests. Docker log trace can be used for DES.

From the collected metrics, the application demonstrates stable performance up to approximately 1200 req/s, beyond which max latencies rise sharply. This behavior indicates the CPU saturating, and seems overloaded at 1600.

Further experiments will explore Docker's CPU and memory cgroup limits (e.g., `--cpus`, `--memory`) and horizontal scaling using multiple app replicas to evaluate how resource allocation affects capacity limits.

Queueing Theory exploration:

For a single-stage service with random arrivals and random service (the classic **M/M/1** model):

- Arrival rate: **λ** (req/s) → Vegeta `-rate`
- Service rate: **μ** (req/s) → how fast the server *could* process if no queue

- Utilization: **$\rho = \lambda/\mu$**

- Mean **response time** (queue + service):
  $W(\lambda) = 1 / (\mu - \lambda)$
- Tail latencies (like p99) grow faster than the mean as the queue forms.

In my measurements:

λ (req/s)	**mean (ms)**	**p99 (ms)**	**max (ms)**
800	1.185	2.528	26.849
1,200	1.287	4.223	24.758
1,600	1.518	6.959	93.737

We model the echo service as a single-stage queue with random arrivals and service. Queueing theory (M/M/1) predicts mean response $W(\lambda)=1/(\mu-\lambda)$, which increases convexly and tends to infinity as the arrival rate λ approaches the service rate μ. In our experiments, as λ increased from 800→1200→1600 req/s, the mean latency rose from 1.185→1.287→1.518 ms, and the p99 latency escalated superlinearly (2.528→4.223→6.959 ms) while the median (p50) remained nearly constant (~1.1–1.24 ms). This pattern—stable median with rapidly growing tail—is the expected signature of queue buildup as utilization approaches saturation (ρ→1). A simple least-squares fit of $1/(\mu-\lambda)$ to the three means gives μ≈2,189 req/s, capturing the observed convex trend. Deviations from the ideal curve are consistent with practical factors (HTTP keep-alive, multi-connection concurrency ⇒ M/M/c effects, and fixed network/stack overhead), but the empirical behavior aligns with queueing predictions.

Furtherwork:

Project Plan:

The Two Possible Directions
Option A — Discrete Event Simulation (DES) with Experimental Data:

We can use our measured Docker load-test data (arrival rates, service times, utilization, latency distributions) to parameterize a discrete-event simulation model. The course currently does DES from theoretical assumptions (Poisson arrivals, exponential service). We can make it data-driven. We could implement an M/M/1 or M/M/c DES that uses our measured λ and μ distributions instead of assumed ones.

- To do: experimenting single thread

Option B — Machine Learning-Assisted Capacity Prediction:
Train a regression or ML model that predicts system performance (latency, utilization, throughput) given parameters like CPU limit, request rate, and replica count. The ML-based capacity planning papers I read used neural networks and statistical models to automate capacity management .

- To do: experimenting with multiple threads

- To do: logging docker trace for collecting sim data. Do des over a single core. Then extend to 2 cores and so on. -thesis B

predictive modeling or ML-based forecasting:

- including **operational analysis** formally:
    - Using **Utilization Law** and **Little's Law** to validate data consistency.
    - Computing **service demand per request** Di=Bi/CiD_i = B_i / C_iDi=Bi/Ci.
    - Cross-checking predicted vs. measured ρ, X, and R.

It'd help me ground my ML regression outputs (like predicting latency) against theoretical consistency checks.

- To do: based on data for cores, predict latency using ML. Predict when I have more threads

These works emphasize predictive automation over analytical modeling.

Conclusion:

References:

[5] S. Taylor and B. Letham, “Forecasting at Scale,” The American Statistician, vol. 72, no. 1, pp. 37–45, 2018.
(Introduces Facebook’s Prophet time-series forecasting model, now used in AWS and Google Cloud monitoring stacks for capacity prediction.)

[6] Amazon Web Services, “AWS Compute Optimizer and Cost Explorer — Right-sizing and Capacity Recommendations,” AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/compute-optimizer/latest/ug/what-is-compute-optimizer.html

[1] P. Kumar, R. Chauhan, and P. Kumar, “Real-Time Monitoring of Servers with Prometheus and Grafana for High Availability,” *Int. Res. J. Eng. Technol. (IRJET)*, vol. 6, no. 4, pp. 6427–6432, Apr. 2019. [Online]. Available: https://www.irjet.net/archives/V6/i4/IRJET-V6I41092.pdf

[2] S. Dhane, P. M. Manwatkar, and A. Wani, “A Survey on Deploying Prometheus and Grafana for Application Monitoring,” *J. Emerg. Technol. Innov. Res. (JETIR)*, vol. 11, no. 1, pp. 409–414, Jan. 2024. [Online]. Available: https://www.jetir.org/papers/JETIR2401076.pdf

[3] M. H. Lubis, A. B. D. Nasution, and A. N. Ritonga, “Enhancing Network Performance Visibility with Grafana and Prometheus: A Case Study at P.T. Nata Digital Solution,” *Int. Conf. Appl. Eng. Technol. (ICANEAT)*, pp. 43–47, 2024. [Online]. Available: https://www.icaneat-apibanyuwangi.org/index.php/icaneat/article/view/43

Integration of Monitoring Tools
Tools like cAdvisor, Prometheus, and Grafana provide comprehensive performance insights.

Your seminar should aim to cover these points:

1. An introduction to the area of your thesis project and its motivation
2. What the state-of-the-art is and what are the open problems? What are the limitations?
3. What problems are you going to tackle in your thesis?
4. What preparation or preliminary work have you done?
5. A plan for your Thesis B and C

Talks:

comp9334 is a course on capacity planning for computer systems and networks that lacks experimental data and measurements to complement all its theories on prediction. So we are thinking of having some platform that could provide the students the opportunity to be able to learn and apply the theories for the predicting performance of various systems with their own personal computers. That would include giving them resources that are accessible and aren't too difficult (time consuming or costly) to set up.

So my literature survey involved the following:

-> looking at modern and popular capacity planning practices in the industry that the students could perhaps emulate.

-> learning capacity planning concepts (resource parameters, workload, operational laws, queueing theory, discrete event simulation). CPU, Memory, workload levels

-> comparing such tools that are used, in terms of cost, accessibility and complexity.

-> looking at teaching practices at other institutions, seeing what they use to help students model real systems and compare them with real data.

Relevant Theories elaboration: ?

Why docker: Docker is a compromised choice, but it is containerised so any modelling done using its resources would not be heavily affected by various overheads or background processes.

Industry practises: So from my findings, modern practises on capacity planning mainly rely on machine learning techniques

Monitoring tools:

Live Container Metrics
cAdvisor collects live CPU, memory, I/O, and network metrics for containers continuously.
Data Storage and Querying
Prometheus stores time-series data and provides powerful querying capabilities for performance analysis.
Performance Visualization
Grafana creates dynamic dashboards to visualize trends and performance metrics effectively.
Load Generation and Testing

Vegeta generates HTTP load simulating varying request rates for controlled performance testing.

-> deciding what capacity to select for applications based on results. (SRE Themed)

-> https://www.unsw.edu.au/canberra/study-with-us/short-courses/reliability-engineering

https://www.unsw.edu.au/canberra/study-with-us/short-courses/reliability-availability-maintainability

Feedback:

-> Identify the difficulties of DES for different setups.

Learn for multithreaded queueing models from a single core.

-> shorten schedule for multicore multithreaded model

Validation of multicore multithreaded model des complementing mlasp paper.